معرفی پروژه

این پروژه یک سیستم مدیریت کاربران است که با Angular بخش فرانت‌اند و ASP.NET Core بخش بک‌اند را پیاده‌سازی کردم. اطلاعات کاربران را در دیتابیس SQL Server ذخیره کردم و ارتباط بین فرانت‌اند و بک‌اند را از طریق API برقرار کردم.

در این پروژه امکان ثبت‌نام و ورود کاربران را پیاده‌سازی کردم تا هر کاربر پس از ورود به حساب خود بتواند اطلاعات پروفایلش را مشاهده، ایجاد، ویرایش و حذف کند. همچنین کاربر می‌تواند پس از ورد از اکانت خود خارج شود.

تکنولوژی‌ها و ابزارهای استفاده‌شده

Frontend

- Angular
- TypeScript
- Reactive Forms
- Angular Routing و Lazy Loading
- Angular Material
- HttpClient
- Angular Signals
- Event Binding و Property Binding
- Dependency Injection
- Signal-based Inputs و Outputs

Backend

- ASP.NET Core
- C#
- Minimal API
- Entity Framework Core (ORM)
- Fluent API

- FluentValidation
- ASP.NET Core Identity
- AutoMapper
- JWT
- Blacklist
- JwtBearerEvents
- برای بررسی و دیباگ توکن‌ها JWT.io

Database

- SQL Server
- برای دیباگ کوئری‌ها و عملیات دیتابیس SQL Server Debugger

ساختار لایه‌های بک‌اند

برای جداسازی مسئولیت‌ها و مدیریت بهتر وابستگی‌ها، بک‌اند پروژه را به پنج لایه تقسیم کردم.

1) Domain : مسئول نگهداری موجودیت‌ها و مفاهیم اصلی برنامه است. این لایه مستقل هست و به سایر لایه‌ها وابستگی ندارد. شامل Entity اطلاعات کاربر و توکن های منقضی شده است.

2) Application : مسئول مدیریت منطق و هماهنگی عملیات برنامه است. این لایه شامل Interfaceها، DTOها، اعتبارسنجی‌ها و Mapping هست و به Domain وابستگی دارد.

3) Infrastructure : مسئول پیاده‌سازی سرویس‌های موردنیاز Application و ارتباط با منابع خارجی است . Repositoryها، Serviceها، DbContext وMigration ها در این لایه قرار دارند. این لایه به Application و Domain وابسته است.

4) Configuration : مسئول مدیریت تنظیمات برنامه است و تنظیمات دیتابیس را از API جدا می‌کند.

5) API : نقطه ورود برنامه و مسئول راه‌اندازی هست. تنظیمات اولیه، Middlewareها وEndpoint ها در این لایه قرار دارند. API از لایه‌های دیگر برای راه‌اندازی و اجرای درخواست‌ها استفاده می‌کند.

جریان اجرای درخواست ثبت‌نام

در فرانت‌اند، کاربر ایمیل و رمز عبور خود را در فرم ثبت‌نام وارد می‌کند. پس از کلیک روی دکمه ثبت‌نام، متد POST در سرویس Angular فراخوانی و HttpClient درخواست ایجاد حساب کاربری را همراه با اطلاعات واردشده به بک‌اند ارسال می‌کند.

در بک‌اند، درخواست واردMiddleware ها شده و به Endpoint مربوط به ثبت‌نام می‌رسد. Endpoint از طریق Dependency Injection به سرویس ثبت‌نام دسترسی پیدا کرده و درخواست را به آن می‌دهد.

در سرویس ثبت‌نام، اطلاعات ورودی را اعتبارسنجی کردم. سپس اطلاعات موردنیاز با استفاده از AutoMapper مپ و از طریق ASP.NET Core Identity حساب کاربری ایجاد کردم. Identity رمز عبور را به‌صورت هش ذخیره می‌کند و از ذخیره شدن رمز عبور خام جلوگیری می‌کند.

پس از ثبت موفق حساب، توکن JWT ایجاد می‌شود. اطلاعات حساب کاربری از طریق Identity و EF Core در دیتابیس ذخیره می‌شوند.

در نهایت نتیجه عملیات از سرویس به Endpoint بازگردانده شده و Endpoint پاسخ HTTP مناسب را به فرانت‌اند ارسال می‌کند.

احراز هویت و مدیریت دسترسی کاربران

پس از ورود موفق کاربر، یک توکن JWT صادر کردم که اطلاعات هویتی کاربر را در قالبClaim ها در خود دارد. این توکن زمان انقضا دارد و با استفاده از Signature از دست‌کاری شدن آن جلوگیری می‌شود.

در درخواست‌های محافظت‌شده، فرانت‌اند توکن را در هدر Authorization و با قالب Bearer به بک‌اند ارسال می‌کند. Middleware احراز هویت، اعتبار توکن را بررسی می‌کند و در صورت معتبر بودن، اطلاعات هویتی کاربر را در اختیار بخش‌های بعدی درخواست قرار می‌دهد.

برای جلوگیری از دسترسی کاربران به اطلاعات یکدیگر، بررسی مالکیت پروفایل را پیاده‌سازی کردم. شناسه کاربر واردشده را از اطلاعات هویتی درخواست استخراج و با شناسه مالک پروفایل مقایسه کردم. در صورت عدم تطابق، اجازه انجام عملیات ویرایش یا حذف داده نمی‌شود.

همچنین برای مدیریت خروج از حساب کاربری، از مکانیزم Blacklist استفاده کردم تا امکان باطل کردن توکن‌های JWT قبل از زمان انقضای آن‌ها فراهم شود. هنگام خروج کاربر، شناسه توکن (Jti) و زمان انقضای آن (Exp) ازClaim های توکن استخراج شده و در دیتابیس ذخیره می‌شوند.

برای جلوگیری از استفاده مجدد از توکن‌های خارج‌شده، در فرآیند اعتبارسنجی JWT از رویداد OnTokenValidated استفاده کردم. در این مرحله، شناسه توکن را استخراج و از طریق سرویس Blacklist بررسی کردم. در صورت وجود توکن در لیست توکن‌های باطل‌شده، اعتبار درخواست رد می‌شود.

Authentication مشخص می‌کند کاربر چه کسی است و Authorization مشخص می‌کند کاربر اجازه انجام چه عملیاتی را دارد.

Validation، Repository و Mapping

FluentValidation

برای اعتبارسنجی اطلاعات ورودی از FluentValidation استفاده کردم. قوانین اعتبارسنجی را از مدل‌ها جدا کردم تا مدیریت و تغییر آن‌ها ساده‌تر باشد. مواردی مثل طول فیلدها، قالب ایمیل و اعتبار اطلاعات ورودی را بررسی کردم.

Repository Pattern

برای جدا کردن منطق دسترسی به داده‌ها از منطق برنامه، از Repository Pattern استفاده کردم. سرویس‌ها از طریق Interface مربوط به Repository به داده‌ها دسترسی دارند و جزئیات ارتباط با دیتابیس را در پیاده‌سازی Repository قرار دادم.

AutoMapper

برای کاهش مپ‌های دستی بینDTO ها و مدل‌های برنامه از AutoMapper استفاده کردم. نگاشت فیلدهای هم‌نام به‌صورت خودکار انجام می‌شود و برای فیلدهای متفاوت نیز امکان ساختاربندی مپ وجود دارد.

Result وResultOfT

برای مدیریت نتیجه عملیات از Result و ResultOfT استفاده کردم. این ساختارها امکان برگرداندن نتیجه موفق یا خطاهای قابل‌انتظار را بدون نیاز به استفاده از Exception برای هر خطا فراهم می‌کنند.

جمع‌بندی

در این پروژه تلاش کردم علاوه بر پیاده‌سازی عملیات مدیریت کاربران، ساختار بک‌اند را به‌صورت لایه‌ای طراحی کنم تا مسئولیت‌های بخش‌های مختلف از یکدیگر جدا باشند.

در طول توسعه پروژه با مفاهیمی مانندDependency Injection ، Repository Pattern،
FluentValidation، AutoMapper، EF Core، ASP.NET Core Identityو JWT کار کردم و کاربرد آن‌ها را در یک پروژه عملی بررسی کردم.

همچنین با پیاده‌سازی احراز هویت و بررسی مالکیت پروفایل‌ها، مدیریت دسترسی کاربران را در نظر گرفتم.